

## HANDS-ON 5

### Module 3: MongoDB — Document Modelling, CRUD & Aggregation

#### Task 1: Create the Collection and Insert Documents

**Goal:** Set up the MongoDB collection and insert realistic feedback documents.

Step 62 & 63: Insert 10 Feedback Documents

```
db.feedback.insertMany([
{
  student_id: 1,
  course_code: "CS101",
  semester: "2022-ODD",
  rating: 5,
  comments: "Excellent teaching.",
  tags: ["challenging","well-structured"],
  submitted_at: new Date("2022-11-30"),
  attachments: [{filename:"notes.pdf",size_kb:240}]
},
{
  student_id: 2,
  course_code: "CS101",
```

```
semester: "2022-ODD",
rating: 4,
comments: "Good examples.",
tags: ["good-examples"],
submitted_at: new Date("2022-11-29"),
attachments: [{filename:"feedback.docx",size_kb:120}]
},
{
  student_id: 3,
  course_code: "CS101",
  semester: "2022-EVEN",
  rating: 5,
  comments: "Very useful course.",
  tags: ["excellent","practical"],
  submitted_at: new Date("2023-04-20"),
  attachments: [{filename:"review.pdf",size_kb:150}]
},
{
  student_id: 4,
  course_code: "CS102",
```

```
semester: "2022-ODD",
rating: 3,
comments: "Average course.",
tags: ["average"],
submitted_at: new Date("2022-12-01"),
attachments: [{filename:"report.pdf",size_kb:180}]
},
{
  student_id: 5,
  course_code: "CS102",
  semester: "2022-EVEN",
  rating: 4,
  comments: "Good content.",
  tags: ["informative"],
  submitted_at: new Date("2023-04-10"),
  attachments: [{filename:"notes.pdf",size_kb:90}]
},
{
  student_id: 6,
  course_code: "EC101",
```

```
semester: "2022-ODD",
rating: 2,
comments: "Needs improvement.",
tags: ["difficult"],
submitted_at: new Date("2022-11-15"),
attachments: [{filename:"remarks.pdf",size_kb:100}]
},
{
  student_id: 7,
  course_code: "ME101",
  semester: "2023-ODD",
  rating: 5,
  comments: "Excellent practical sessions.",
  tags: ["practical","excellent"],
  submitted_at: new Date("2023-11-01"),
  attachments: [{filename:"lab.pdf",size_kb:300}]
},
{
  student_id: 8,
  course_code: "CS103",
```

```
semester: "2022-ODD",
rating: 4,
comments: "Well organized.",
tags: ["organized"],
submitted_at: new Date("2022-11-20"),
attachments: [{filename:"review.doc",size_kb:110}]
},
{
  student_id: 9,
  course_code: "CS103",
  semester: "2023-EVEN",
  rating: 3,
  comments: "Moderately useful.",
  tags: ["moderate"],
  submitted_at: new Date("2023-05-15")
},
{
  student_id: 10,
  course_code: "EC101",
  semester: "2023-ODD",
```

```
rating: 1,  
comments: "Very difficult course.",  
tags: ["hard"],  
submitted_at: new Date("2023-10-10"),  
attachments: [{filename:"feedback.pdf",size_kb:95}]  
}  
])
```

## Step 64: Verify Inserts

### CODE

```
db.feedback.countDocuments()
```

### EXPECTED OUTPUT

10

### COMMENT

```
// Task 1 Completed
```

```
// Database: college_nosql
```

```
// Collection: feedback
```

```
// Inserted 10 feedback documents
```

```
// One document intentionally omits attachments
```

```
// countDocuments() returned 10
```

## Task 2: CRUD Operations

65. Find all feedback with rating 5

JAVA SCRIPT

```
db.feedback.find({ rating: 5 })
```

66. Find CS101 feedback with tag "challenging"

```
db.feedback.find({  
  course_code: "CS101",  
  tags: "challenging"  
})
```

7. Projection (student\_id, course\_code, rating only)

```
db.feedback.find(  
  {},  
  {  
    student_id: 1,  
    course_code: 1,  
    rating: 1,  
    _id: 0  
  }  
)
```

68. Add needs\_review:true where rating < 3

```
db.feedback.updateMany(  
  { rating: { $lt: 3 } },
```

```
{ $set: { needs_review: true } }  
)
```

#### 69. Push "reviewed" tag

```
db.feedback.updateMany(  
  { needs_review: true },  
  { $push: { tags: "reviewed" } }  
)
```

#### 70. Delete documents from 2021-EVEN

```
db.feedback.deleteMany({  
  semester: "2021-EVEN"  
})
```

### **Task 3: Aggregation Pipeline**

#### 71. Average rating and count by course

```
db.feedback.aggregate([  
  {  
    $match: {  
      semester: "2022-ODD"  
    }  
  },  
  {  
    $group: {
```



```

        _id: "$course_code",
        avg_rating: { $avg: "$rating" },
        total_feedback: { $sum: 1 }
    }
},
{
    $sort: {
        avg_rating: -1
    }
}
])

```

## 72. Add project and round average rating

```

db.feedback.aggregate([
{
    $match: {
        semester: "2022-ODD"
    }
},
{
    $group: {
        _id: "$course_code",
        avg_rating: { $avg: "$rating" },
        total_feedback: { $sum: 1 }
    }
},

```

```

{
  $project: {
    _id: 0,
    course_code: "$_id",
    average_rating: {
      $round: ["$avg_rating", 1]
    },
    total_feedback: 1
  }
},
{
  $sort: {
    average_rating: -1
  }
}
])

```

### 73. Tag Frequency Leaderboard

```

db.feedback.aggregate([
  {
    $unwind: "$tags"
  },
  {
    $group: {
      _id: "$tags",
      count: { $sum: 1 }
    }
  }
])

```

```
    }  
  },  
  {  
    $sort: {  
      count: -1  
    }  
  }  
])
```

## 74. Create Index and Verify

Create index:

```
db.feedback.createIndex({  
  course_code: 1  
})
```

Verify:

```
db.feedback.find({  
  course_code: "CS101"  
}).explain("executionStats")
```

### Comment

```
// Task 2 & Task 3 Completed
```

```
// CRUD operations written successfully.
```

```
// Aggregation pipelines created.
```

```
// Index added on course_code.
```

// Expected query plan uses IXSCAN instead of COLLSCAN.